Pipelined CPU and Cache Experiment

By Divya Dosetty

What was compared

Three bare-metal MIPS programs (`loop`, `arraysum` and `factorial`) were run on gem5's pipelined out-of-order CPU, `MipsO3CPU`, in two configurations: the default L1 data cache, and an 8 kB L1 data cache (`--l1d_size=8kB`). The pipelined results were compared with the simple CPU results (TimingSimpleCPU) from Chinmay, which I also reproduced independently for loop and factorial.

`MipsO3CPU` has the stages Fetch, Decode, Rename, Issue/Execute/Writeback and Commit. `MipsMinorCPU` was tried first but gave wrong results in this gem5 version (it stopped after 4 instructions), so it was not used.

Results

Program	Pipelined cycles	Pipelined, small cache cycles	Change
loop	2,000,593	2,000,593	none
arraysum	1,037,671	1,859,727	+79% (about 1.79x slower)
factorial	2,100,778	2,100,778	none

The instruction counts were identical with both caches (loop 5,000,013; arraysum 2,040,215; factorial 5,700,010), so only the timing changed.

Instructions per cycle (IPC = simInsts ÷ cycles):

Program	IPC, normal cache	IPC, small cache
loop	2.50	2.50
arraysum	1.97	1.10
factorial	2.71	2.71

IPC uses the pipelined CPU's own instruction counts, which exclude these NOPs.

Pipeline benefit

The pipelined CPU completes more than one instruction per cycle on all three programs (IPC of 2.50 for `loop`, 1.97 for `arraysum` and 2.71 for `factorial`), which a simple non-pipelined CPU cannot do. `factorial` has the highest IPC.

Program	Simple CPU cycles	Pipelined cycles	Speedup	Pipelined, 8 kB cache	Speedup, 8 kB cache
loop	14,000,560	2,000,593	7.00x	2,000,593	7.00x
arraysum	5,168,492	1,037,671	4.98x	1,859,727	2.78x
factorial	15,600,552	2,100,778	7.43x	2,100,778	7.43x

Speedup = simple-CPU cycles ÷ pipelined cycles. Simple-CPU numbers are from Chinmay's baseline runs, which I reproduced on my side.

The program that benefited most from the pipeline is factorial (7.43x fewer cycles than the simple CPU), followed by loop (7.00x) and arraysum (4.98x). arraysum gained least because it spends more time waiting on memory, which the pipeline cannot hide. With the 8 kB data cache its speedup drops further, to 2.78x.

Note on instruction counts. For loop and factorial, the simple CPU reports 1,000,001 more instructions than the pipelined CPU (loop: 6,000,014 vs 5,000,013; factorial: 6,700,011 vs 5,700,010). arraysum differs by only 1 (2,040,216 vs 2,040,215). Disassembly with objdump shows the compiler places a nop in the delay slot after each loop branch, which adds one instruction per iteration over 1,000,000 iterations. The simple CPU counts these NOPs and the pipelined CPU does not. Both CPUs ran the same compiled programs, and the simple-CPU results were reproduced independently by two people. The comparison therefore uses cycles, which are not affected by this difference.

```
--
  400190:       8ca301b0        lw      v1,432(a1)
  400194:       0062182a        slt     v1,v1,v0
  400198:       1060fffb        beqz    v1,400188 <main+0x18>
  40019c:       00000000        nop
  4001a0:       3c020041        lui     v0,0x41
  4001a4:       ac4401c0        sw      a0,448(v0)
  4001a8:       03e00008        jr      ra
@dosettydivya →/workspaces/gem5-pipeline-project/gem5_project_kit/shared (main) $
```

Figure: objdump output of loop. The beqz branch at 400198 is followed by a nop at 40019c, in the branch delay slot.

Cache effect

`arraysum` was the only program affected by the smaller cache. Its cycle count rose from 1,037,671 to 1,859,727 (about 79% more) and its IPC fell from 1.97 to 1.10. It reads through an array, so with only 8 kB of data cache many more accesses miss and have to wait for slower memory.

`loop` and `factorial` use almost no data memory, so their cycle counts stayed exactly the same. This is expected and does not mean the setting failed. The `arraysum` result shows the 8 kB option was applied.

Conclusion

The pipelined CPU was about 5x to 7.4x faster than the simple CPU, with factorial gaining most (7.43x) and arraysum least (4.98x). Cache size only matters for programs that use a lot of memory: the 8 kB cache cut arraysum's speedup from 4.98x to 2.78x, while loop and factorial were unaffected.

Notes

`o3small_arraysum` was rerun separately with gem5's remote debug port turned off (`--remote-gdb-port=0`), because the first attempt hung waiting on a debugger connection. Results are pushed to `results/divya` in the group repo.